How to train the language models

This contains a compilation of some thoughts I have around training language models to predict the keep/remove decisions.

We've already trained logistic regression and XGBoost-based models, across a few ablations. Now I'd like to see how language models would fare.

Some variants I'm thinking of include:

- One-shot prompting:
  - With a small model
  - With a large model
- Few-shot prompting:
  - With a small model
  - With a large model
- ModernBERT
- LoRA fine-tuning

I'll also do the prompting ones across two sets of inputs:

- Just the original post
- Original post + mirrored post

We showed users both the original post and the mirrored post as part of the linked fate procedure, and we saw a significant difference in keep/remove decisions based on whether users were in the linked fate condition or if they only saw the original post. However, when we trained our predictive models, we saw that including only the original post was just as predictive as showing both the original and mirrored posts, suggesting that it was the mere presence of the mirrored post that affected whether users chose to keep or remove. We also observed that users in our training condition (where they saw the linked fate procedure for the first half of the study, and only the original posts in the second half of the study) also matched the performance of users in the linked-fate-only condition, censoring outgroup posts less and ingroup posts more as compared to users in the control condition (i.e., only saw the original posts). All of this suggests that the key driver is the mere presence or awareness of the linked fate procedure (i.e., seeing the mirrored posts).

For completeness, we can do some initial experiments that include both the original post and the original post plus the mirrored post. However, I expect those to have non-significant results due to our previous findings suggesting that the mere presence of the mirror, rather than anything about the substance of the mirrored post itself, drove performance. If we replicate that, then we'll continue on training the ModernBERT and LoRA-tuned models with just the original text.

Experiment 1: Prompting

For the prompting experiments, we'll do them in models/llm_api/{one_shot/few_shot}/{original/original_plus_

Each will have a prompt.py and a train.py, for consistency with the other approaches.

For the models, we'll use:

- small: gpt-5.4 nano
- large: gpt-5.5

We'll run these and report the results as a table here, with columns:

- type: one-shot/few-shot
- ablation: original, original plus mirror
- model size: small, large
- model name
- split: train/test
- accuracy, other metrics...

Prompting results (Experiment 1)

We only completed full-dataset runs for the two one-shot / small (`gpt-5.4-nano`) ablations (original-only and original-plus-mirror). Each run scored the full 80/20 split (`n_train=7032`, `n_test=1759`; 8,791 API requests per variant). We did not run the remaining six variants (one-shot/large, few-shot/{original,original_plus_mirror}×{small,large}) for cost reasons; the two completed runs are enough of a prompting baseline to compare against the embedding classifiers and to decide whether to invest in ModernBERT / LoRA next.

Precision, recall, F1, ROC-AUC, and PR-AUC use `remove` as the positive class (`y=1`).

type	ablation	model_size	model_name	split	accuracy	precision	recall	f1	roc_auc	pr_auc
one-shot	original	small	gpt-5.4-nano	train	0.690	0.519	0.443	0.478	0.678	0.491
one-shot	original	small	gpt-5.4-nano	test	0.686	0.510	0.485	0.497	0.695	0.515
one-shot	original plus mirror	small	gpt-5.4-nano	train	0.682	0.504	0.308	0.382	0.627	0.448
one-shot	original plus mirror	small	gpt-5.4-nano	test	0.676	0.490	0.314	0.383	0.615	0.448

On the test set, original-only one-shot prompting slightly outperforms original-plus-mirror (accuracy 0.686 vs 0.676; F1 0.497 vs 0.383), consistent with the embedding ablations where adding mirror text did not help. Absolute performance is in the same ballpark as logistic regression on original-only embeddings (test accuracy ~0.67, F1 ~0.55), so prompting alone is not a clear win over the cheaper embedding baselines.

Experiment 2: ModernBERT

We also want to use ModernBERT to develop a fine-tuned classifier for our task. ModernBERT is an improvement over the original BERT model, with additions like RoPE, FlashAttention, and other techniques from language model research. The 8,192-token context window is a big practical improvement over classic BERT's usual 512-token limit.

HuggingFace natively supports ModernBERT. In addition, we can use AWS Sagemaker as our trainer in order to avoid having to do local computation. We'll use the base model of ModernBERT, not the large one.

We'd like to use the following:

- Weights and Biases for ML training logging.
- HuggingFace for the interface.
- AWS Sagemaker for the compute.

We'll use the same training data that we used for our other ML models (where each row is 1 post and label). We'll use only the original text, rather than the original text plus the mirrored text.

We'll collect the following metrics:

- Accuracy
- Precision
- Recall
- F1

We'll use cross-entropy loss with 2 labels. Because of our class imbalance, we'll use weighted loss, where we multiply the loss by the class weight. We want to upweight the minority class (`remove=1`) so we prioritize getting those correct. We'll double the loss for `remove=1`. This also lets us improve recall for `remove` labels, but we'll want to be careful as we also don't want many false positives (which worsens our precision).

Our prediction task is "predict remove", so we need to transform the training data to create a binary label.

We'll develop in the following order:

Step 1: Head-only training (frozen encoder)

Fine-tune ModernBERT on the training data and evaluate training curves. We'll evaluate the training curves to review for overfitting.

We'll do head-only training for our task, as our training dataset is pretty small and we want something that is somewhat able to work out-of-distribution and avoid catastrophic forgetting.

Step 2: Calibration

Given our outputs, we can evaluate calibration curves varying the binary thresholds from p=0.1 to p=0.9, in increments of 0.1.

More implementation details

We'll store all the work in experiments/predict_keep_remove_2026_07_01/models/modernbert/

We'll use the following file structure:

```
experiments/
  predict_keep_remove_2026_07_01/
    dataloader.py                  # existing raw dataframe loader

    models/
      modernbert/
        README.md
        dataloader.py              # wraps parent dataloader, converts labels
        train.py                   # local/SageMaker-compatible training entrypoint
        evaluate.py                # optional test-set evaluation
        predict.py                 # optional inference helper
        launch_sagemaker.py        # submits remote SageMaker job
        requirements.txt
        configs/
          modernbert_base.yaml
        artifacts/
          .gitkeep                 # local outputs ignored by git
```

For packages, add to requirements.txt and then also add to the root pyproject.toml as optional dependencies titled "modernbert-training".

We can use something like this as the config:

```yaml
# experiments/predict_keep_remove_2026_07_01/models/modernbert/configs/modernbert_base.yaml

model_name: answerdotai/ModernBERT-base

text_col: text
label_col: label
```

```yaml
max_length: 256
learning_rate: 2e-5
num_train_epochs: 10 # let's run for 10 epochs
per_device_train_batch_size: 8
per_device_eval_batch_size: 16
weight_decay: 0.01

output_dir: artifacts/modernbert-base
random_state: 42
```

For ~2,000 posts and 10 epochs, we can use a smaller GPU setup:

- Instance: ml.g4dn.xlarge
- GPU: 1 × NVIDIA T4, 16 GB GPU memory
- Estimated SageMaker on-demand price: about $0.74/hr in us-east-2
- Expected runtime: ~5–20 minutes
- Expected compute cost: usually <$0.25, likely closer to $0.05–$0.15

Let's use `us-east-2` for our AWS setup.

For our data splits, we want to do a 80/10/10 split between train/validation/test.

ModernBERT results (Experiment 2)

Full 10-epoch head-only fine-tune of `answerdotai/ModernBERT-base` on original post text only (`freeze_encoder=true`, class weights keep=1.0 / remove=2.0, max_length=256). Trained on SageMaker `ml.g4dn.xlarge` in `us-east-2` (job `modernbert-keep-remove-2026-07-03-21-09-05-077`; run id `2026_07_03-210901`). Split sizes: `n_train=7032`, `n_val=879`, `n_test=880`. Metrics below use decision threshold `0.5` with `remove` as the positive class (`y=1`).

Artifacts: `experiments/predict_keep_remove_2026_07_01/models/modernbert/artifacts/modernbert-b` (`metrics.json`, predictions, `calibration.json`). S3: `s3://jspsych-mirror-view-4/modernbert-training/` W&B: modernbert-base-2026_07_03-21:18:33.

model	freeze_encoder	split	accuracy	precision	recall	f1	roc_auc	pr_auc
ModernBERT-base	true	train	0.691	0.515	0.585	0.548	0.719	0.545
ModernBERT-base	true	val	0.681	0.502	0.552	0.525	0.693	0.525
ModernBERT-base	true	test	0.694	0.520	0.596	0.555	0.742	0.569

On the test set, head-only ModernBERT reaches accuracy 0.694 and F1 0.555, slightly above one-shot original-only prompting (accuracy 0.686, F1 0.497) and in line with the logistic-regression embedding baseline (test accuracy ~0.67, F1

~0.55). Train and val metrics are close to test (no large train–test gap), consistent with a frozen encoder and a small trainable head (1,538 parameters). Threshold calibration over `0.1`–`0.9` is in `calibration.json` for follow-up threshold selection.

ModernBERT threshold tuning

Test-set threshold sweep (`0.1`–`0.9`, step `0.1`) on run `2026_07_03-210901`. Artifacts: `models/modernbert/outputs/threshold_analysis/2026_07_05-13:57:45/`.

metric	highest value	value at p=0.5	threshold at highest
accuracy	0.726	0.694	0.6
precision	0.741	0.520	0.8
recall	0.996	0.596	0.1
f1	0.578	0.555	0.4

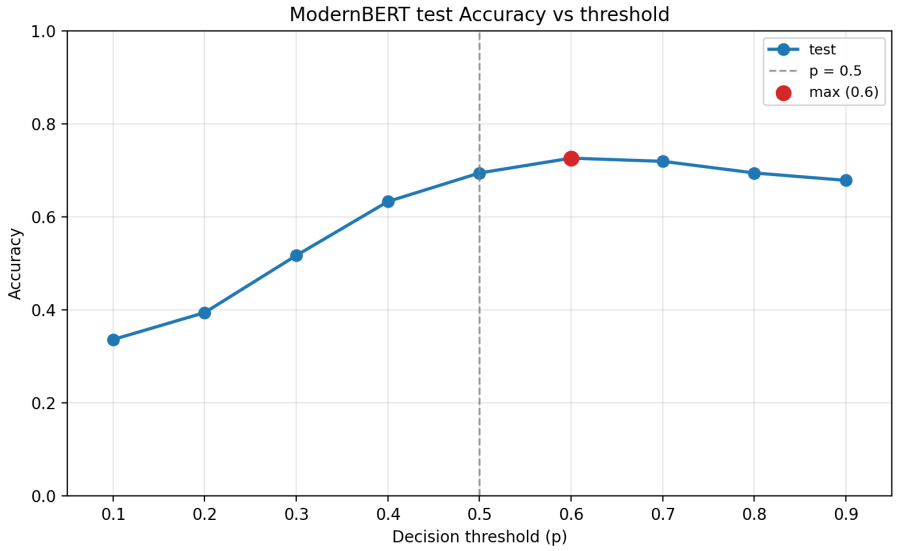

Figure 1: ModernBERT test accuracy vs threshold

Experiment 3: LoRA-tuned models

...

We'll use a Qwen model. We'll use 2 variants of Qwen models. We'll use AWS Bedrock for training.

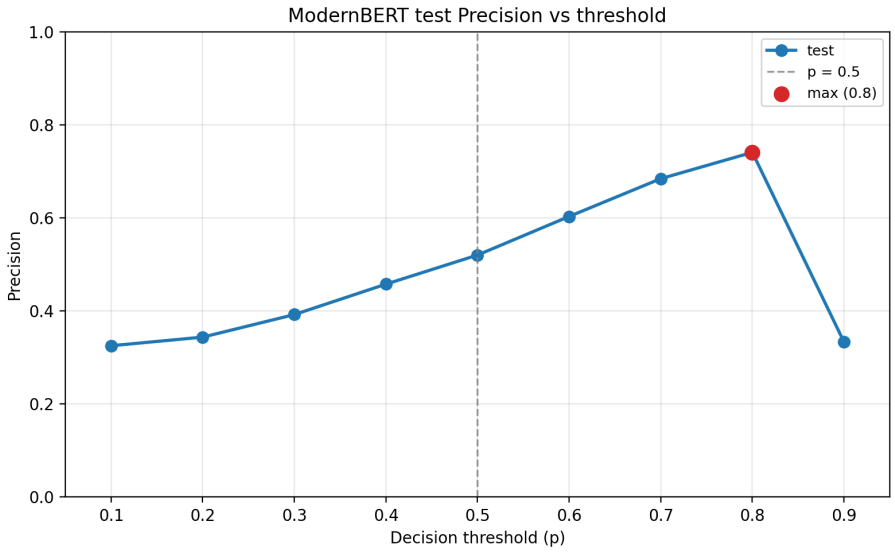

Figure 2: ModernBERT test precision vs threshold

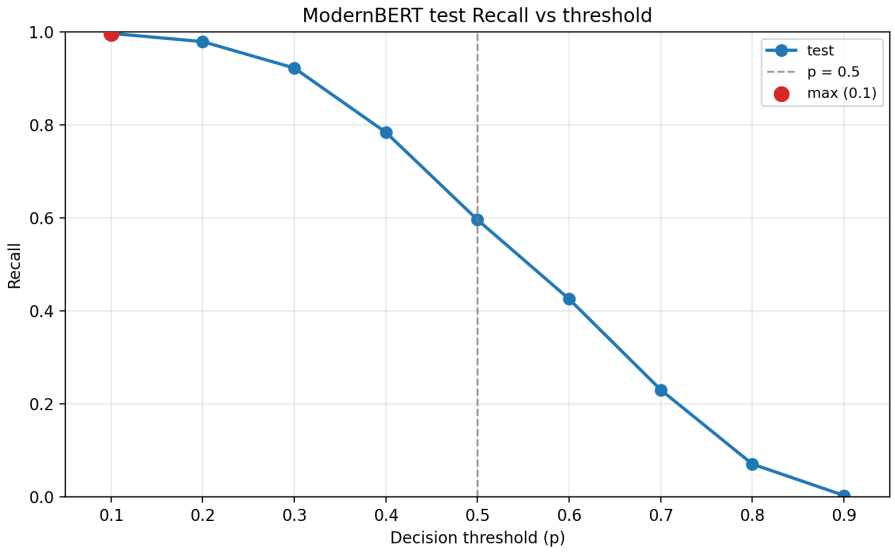

Figure 3: ModernBERT test recall vs threshold

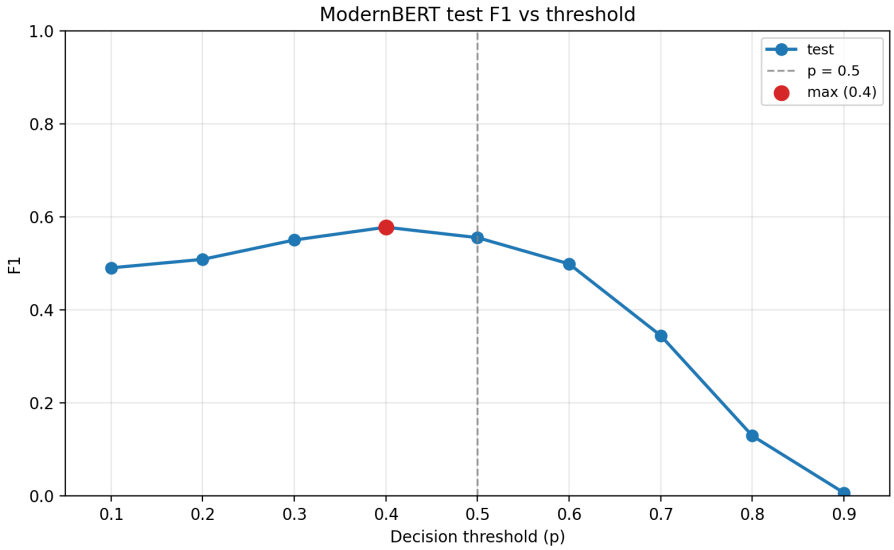

Figure 4: ModernBERT test F1 vs threshold